JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZoneOffsetTransitionRule

java.lang.Object
    java.time.zone.ZoneOffsetTransitionRule

All Implemented Interfaces:

Serializable

```
public final class ZoneOffsetTransitionRule
extends Object
implements Serializable
```

A rule expressing how to create a transition.

This class allows rules for identifying future transitions to be expressed. A rule might be written in many forms:

- the 16th March
- the Sunday on or after the 16th March
- the Sunday on or before the 16th March
- the last Sunday in February

These different rule types can be expressed and queried.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static enum	ZoneOffsetTransitionRule	A definition of the way a local time can be converted to the actual transition date-time.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description

ZoneOffsetTransition	**createTransition**(int year)	Creates a transition instance for the specified year.
boolean	**equals**(Object otherRule)	Checks if this object equals another.
int	**getDayOfMonthIndicator**()	Gets the indicator of the day-of-month of the transition.
DayOfWeek	**getDayOfWeek**()	Gets the day-of-week of the transition.
LocalTime	**getLocalTime**()	Gets the local time of day of the transition which must be checked with isMidnightEndOfDay().
Month	**getMonth**()	Gets the month of the transition.
ZoneOffset	**getOffsetAfter**()	Gets the offset after the transition.
ZoneOffset	**getOffsetBefore**()	Gets the offset before the transition.
ZoneOffset	**getStandardOffset**()	Gets the standard offset in force at the transition.
ZoneOffsetTransitionI	**getTimeDefinition**()	Gets the time definition, specifying how to convert the time to an instant.
int	**hashCode**()	Returns a suitable hash code.
boolean	**isMidnightEndOfDay**()	Is the transition local time midnight at the end of day.
static ZoneOffsetTransitionI	**of**(Month month, int dayOfMonthIndicator,	Obtains an instance defining the yearly rule

	DayOfWeek dayOfWeek, LocalTime time, boolean timeEndOfDay, ZoneOffsetTransitionRule ZoneOffset standardOffse ZoneOffset offsetBefore, ZoneOffset offsetAfter)	to create transitions between two offsets.
String	**toString**()	Returns a string describing this object.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Details

of

```
public static ZoneOffsetTransitionRule of
(Month month,
 int dayOfMonthIndicator,
 DayOfWeek dayOfWeek,
 LocalTime time,
 boolean timeEndOfDay,
 ZoneOffsetTransitionRule.TimeDefinition timeDefinition,
 ZoneOffset standardOffset,
 ZoneOffset offsetBefore,
 ZoneOffset offsetAfter)
```

Obtains an instance defining the yearly rule to create transitions between two offsets.

Applications should normally obtain an instance from ZoneRules. This factory is only intended for use when creating ZoneRules.

Parameters:

`month` - the month of the month-day of the first day of the cutover week, not null

`dayOfMonthIndicator` - the day of the month-day of the cutover week, positive if the week is that day or later, negative if the week is that day or earlier, counting from the last day of the month, from -28 to 31 excluding 0

`dayOfWeek` - the required day-of-week, null if the month-day should not be changed

`time` - the cutover time in the 'before' offset, not null

`timeEndOfDay` - whether the time is midnight at the end of day

`timeDefinition` - how to interpret the cutover

`standardOffset` - the standard offset in force at the cutover, not null

`offsetBefore` - the offset before the cutover, not null

`offsetAfter` - the offset after the cutover, not null

Returns:

the rule, not null

Throws:

`IllegalArgumentException` - if the day of month indicator is invalid

`IllegalArgumentException` - if the end of day flag is true when the time is not midnight

`IllegalArgumentException` - if `time.getNano()` returns non-zero value

getMonth

`public Month getMonth()`

Gets the month of the transition.

If the rule defines an exact date then the month is the month of that date.

If the rule defines a week where the transition might occur, then the month if the month of either the earliest or latest possible date of the cutover.

Returns:

the month of the transition, not null

getDayOfMonthIndicator

`public int getDayOfMonthIndicator()`

Gets the indicator of the day-of-month of the transition.

If the rule defines an exact date then the day is the month of that date.

If the rule defines a week where the transition might occur, then the day defines either the start of the end of the transition week.

If the value is positive, then it represents a normal day-of-month, and is the earliest possible date that the transition can be. The date may refer to 29th February which should be treated as 1st March in non-leap years.

If the value is negative, then it represents the number of days back from the end of the month where `-1` is the last day of the month. In this case, the day identified is the latest possible date that the transition can be.

Returns:

the day-of-month indicator, from -28 to 31 excluding 0

getDayOfWeek

```
public DayOfWeek getDayOfWeek()
```

Gets the day-of-week of the transition.

If the rule defines an exact date then this returns null.

If the rule defines a week where the cutover might occur, then this method returns the day-of-week that the month-day will be adjusted to. If the day is positive then the adjustment is later. If the day is negative then the adjustment is earlier.

Returns:

the day-of-week that the transition occurs, null if the rule defines an exact date

getLocalTime

```
public LocalTime getLocalTime()
```

Gets the local time of day of the transition which must be checked with `isMidnightEndOfDay()`.

The time is converted into an instant using the time definition.

Returns:

the local time of day of the transition, not null

isMidnightEndOfDay

```
public boolean isMidnightEndOfDay()
```

Is the transition local time midnight at the end of day.

The transition may be represented as occurring at 24:00.

Returns:

whether a local time of midnight is at the start or end of the day

getTimeDefinition

`public ZoneOffsetTransitionRule.TimeDefinition getTimeDefinition()`

Gets the time definition, specifying how to convert the time to an instant.

The local time can be converted to an instant using the standard offset, the wall offset or UTC.

Returns:

the time definition, not null

getStandardOffset

`public ZoneOffset getStandardOffset()`

Gets the standard offset in force at the transition.

Returns:

the standard offset, not null

getOffsetBefore

`public ZoneOffset getOffsetBefore()`

Gets the offset before the transition.

Returns:

the offset before, not null

getOffsetAfter

`public ZoneOffset getOffsetAfter()`

Gets the offset after the transition.

Returns:

the offset after, not null

createTransition

`public ZoneOffsetTransition createTransition(int year)`

Creates a transition instance for the specified year.

Calculations are performed using the ISO-8601 chronology.

Parameters:

year - the year to create a transition for, not null

Returns:

the transition instance, not null

equals

```
public boolean equals(Object otherRule)
```

Checks if this object equals another.

The entire state of the object is compared.

Overrides:

equals in class Object

Parameters:

otherRule - the other object to compare to, null returns false

Returns:

true if equal

See Also:

Object.hashCode(), HashMap

hashCode

```
public int hashCode()
```

Returns a suitable hash code.

Overrides:

hashCode in class Object

Returns:

the hash code

See Also:

Object.equals(java.lang.Object),
System.identityHashCode(Object)

toString

```
public String toString()
```

Returns a string describing this object.

Overrides:

toString in class Object

Returns:

a string for debugging, not null

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.